
目录

- 第 1 章 设计任务于要求..... 2
 - 1.1 设计任务 2
 - 1.2 设计要求 2
 - 1.3 设计步骤 2
- 第 2 章 仿真..... 2
 - 2.1 实验原理 2
 - 2.2 仿真测试结果 5
- 第 3 章 硬件设计..... 7
 - 3.1 驱动部分 7
 - 3.2 控制部分 12
 - 3.3 PCB Layout 14
- 第 4 章 软件设计..... 15
 - 4.1 系统配置 16
 - 4.2 控制代码 20
- 第 5 章 测试结果分析..... 30
- 第 6 章 设计总结..... 32

第1章 设计任务于要求

1.1 设计任务

主要任务：

- 1.基于 PI 的有感无刷直流电机速度闭环控制系统仿真
- 2.设计并制作有感无刷直流电机驱动模块
- 3.采用单片机实现有感无刷直流电机的速度控制。

1.2 设计要求

仿真：额度速度 1500rpm，其它参数自定义

驱动模块：控制对象直流无刷电机外形及参数如图 1 所示。工作电压：24VDC。带工作指示灯。输入输出配置接线端子，方便与电源、电机控制器及信号连接。

根据题目要求进行方案论证，理论分析与计算，可重新或修改参考设计。

1.3 设计步骤

借助 Matlab 软件,进行基于 PI 的有感无刷直流电机速度闭环控制系统仿真。

借助嘉立创 EDA 软件,设计电路原理图以及 PCB 的布局布线模块化规范化美观整齐。PCB 上需丝印“武汉纺织大学+电气 1220x+本人姓名” logo。

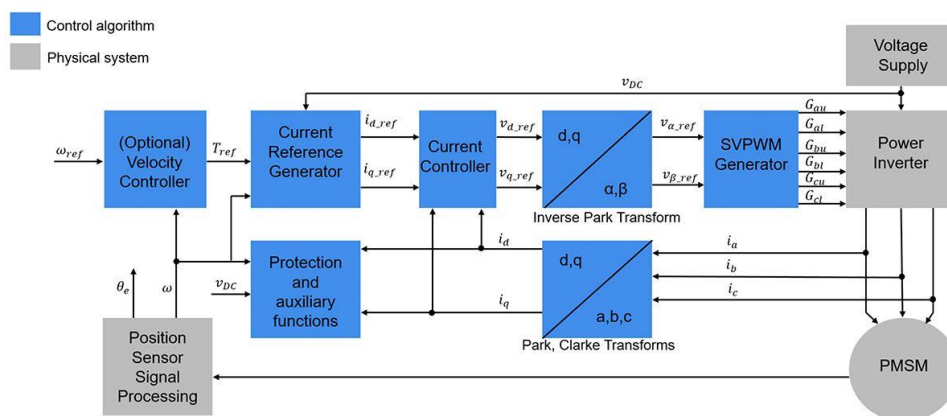
通过嘉立创平台将 PCB 制作出来，焊接元器件，调试实现无刷直流电机的速度控制功能。

撰写设计报告，包含 Matlab 系统仿真图、电路设计图（原理图和 PCB）、电路板正反面照片等等内容。

第2章 仿真

2.1 实验原理

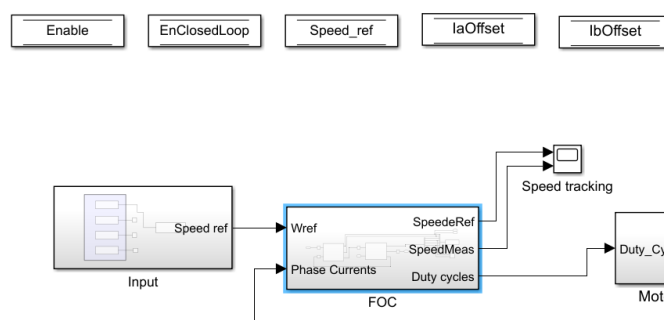
PMSM/BLDC 的 FOC 经典控制框图如下



图表 1 FOC 控制框图

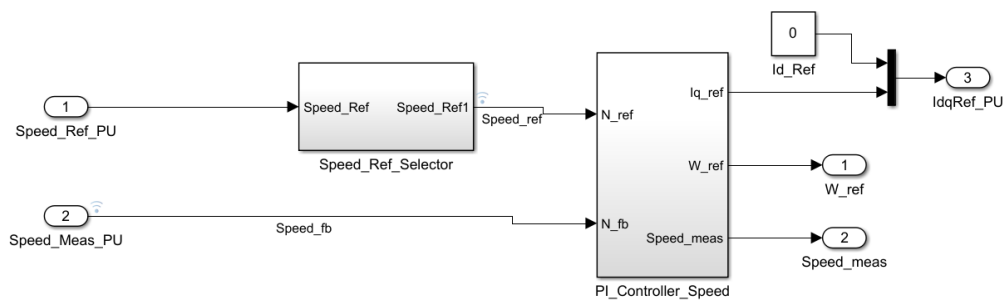
FOC（磁场定向控制）的核心思想是：通过 **Clarke 和 Park 变换**将三相定子电流转换到以转子磁场为基准的 $d-q$ 坐标系，在该坐标系中 i_d 控制磁通， i_q 控制转矩，实现二者解耦，使交流电机的控制特性类似直流电机；然后通过 PI 控制器调节 i_d, i_q ，再逆变换生成三相电压驱动电机，从而实现高效、平滑、快速的电机转矩与速度控制。

将各种模块封装，总体模型框图如下，使用 Offset 代替 HALL 进行电角度估算。



图表 2 总体框图

FOC 框图中主要包含了一个带有 PI 调节器的速度控制器，可以通过编码器或者 HALL 传感器简单获取转速，在无感控制算法中可以通过一些观测器或者反电动势以及高频注入等获取。



图表 3 速度控制器

在 FOC 控制算法中，**电流控制环节是核心部分**。该模块主要包括 **Clarke 变换、Park 变换及其逆变换**。流程如下：

1. 电流采集：通过硬件电流采样电路获取电机三相电流，可采用三相全采样（高侧/低侧电阻）、两相采样并推算第三相电流。在仿真环境中通常直接提供三相电流。

2. 坐标变换：

首先对三相电流进行 Clarke 变换，得到两相静止坐标系 i_α, i_β ；

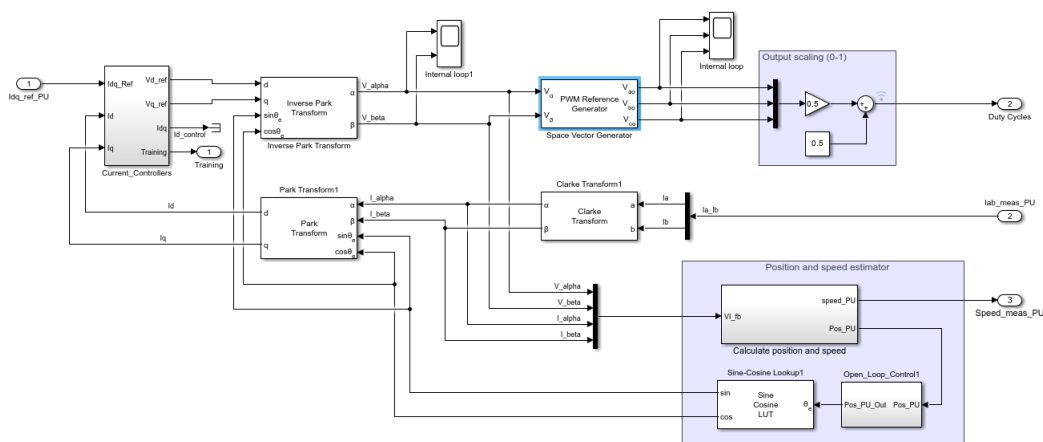
再通过 Park 变换，结合转子位置信息，将其转换到旋转坐标系 (i_d, i_q) 。

3. 电流控制：将 (i_d, i_q) 与参考值比较，通过 PI 控制器得到电压指令 (v_d, v_q) 。

4. 逆坐标变换与调制：

将 (v_d, v_q) 经过 逆 Park 变换得到 v_α, v_β ；

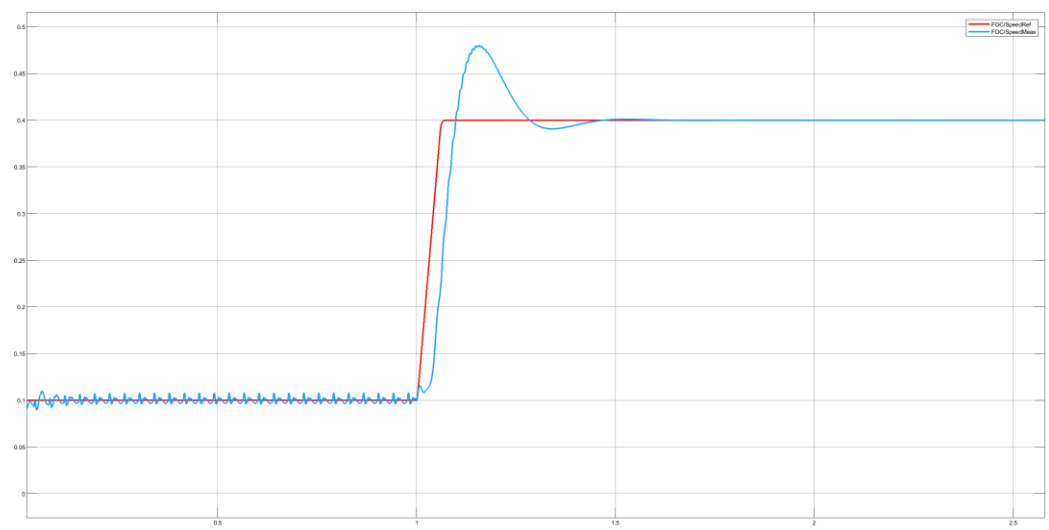
再通过 空间矢量 PWM (SVPWM) 模块生成三相电压指令 (v_a, v_b, v_c) ，并映射到逆变器的八个开关状态，最终得到对应的 MOSFET 栅极驱动信号。



图表 4 电流控制模块

2.2 仿真测试结果

速度跟随测试结果如下

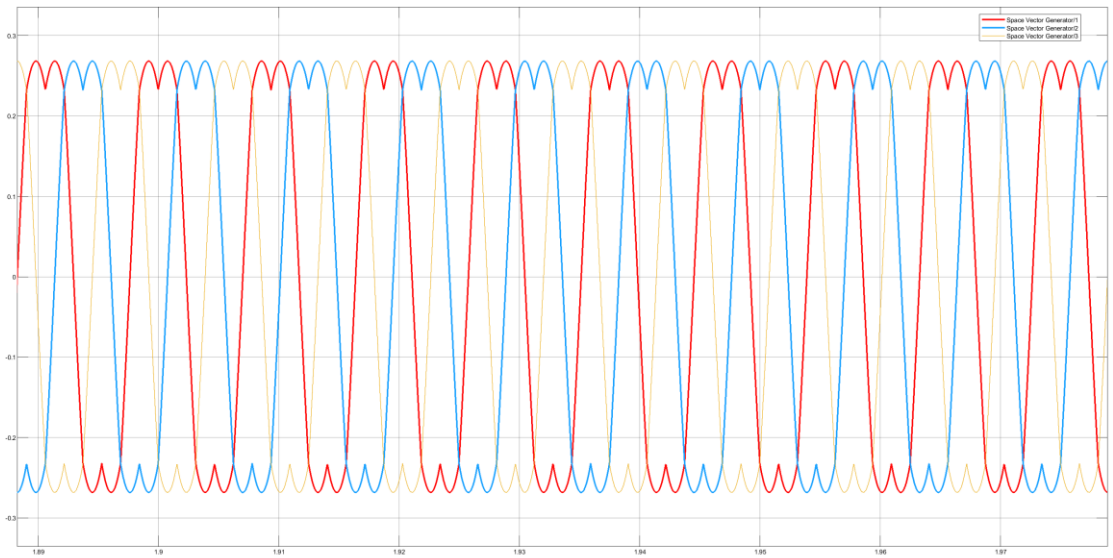


图表 5 速度跟随控制

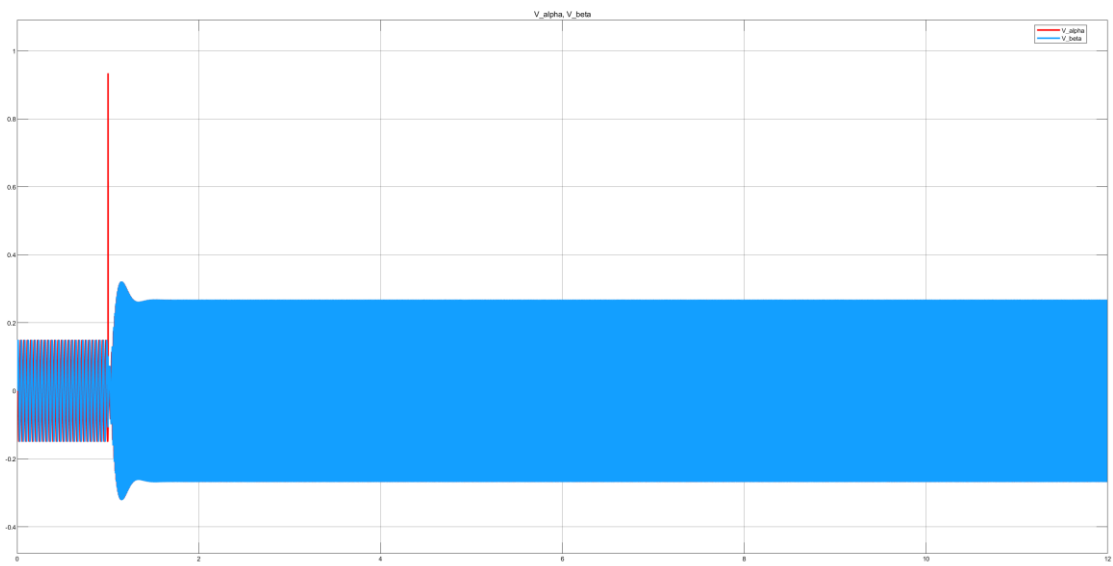
在 SVPWM 调制下，由于零矢量的引入，波形应该是类似马鞍波形，并且三相相位差距 120° 。



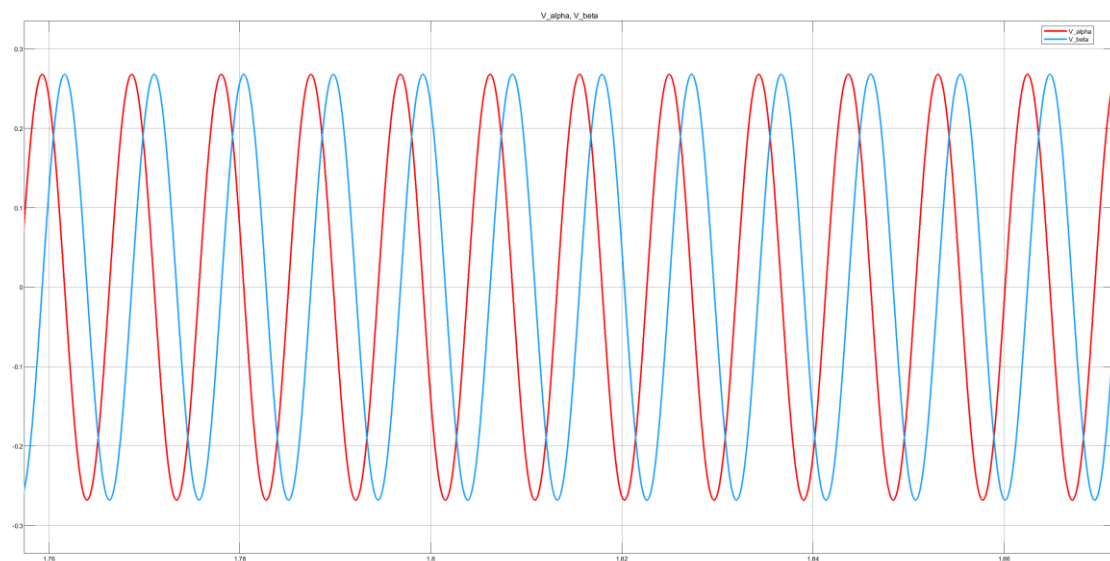
图表 6 三相电压



图表 7 波形细节



图表 8 两相静止坐标系



图表 9 波形细节

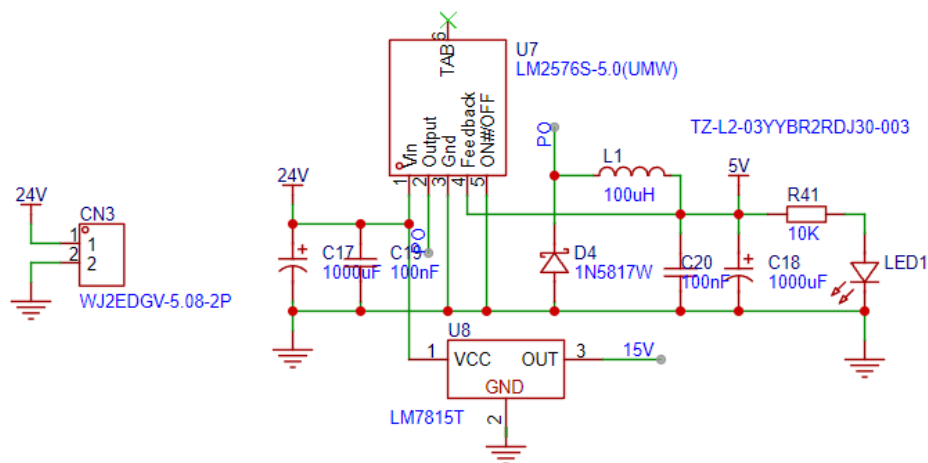
第3章 硬件设计

本次硬件的设计分为两大部分，1. 驱动部分 2. 控制部分，其中驱动部分负责采样以及三相逆变部分，控制部分主要负责计算并且输出开关量。

3.1 驱动部分

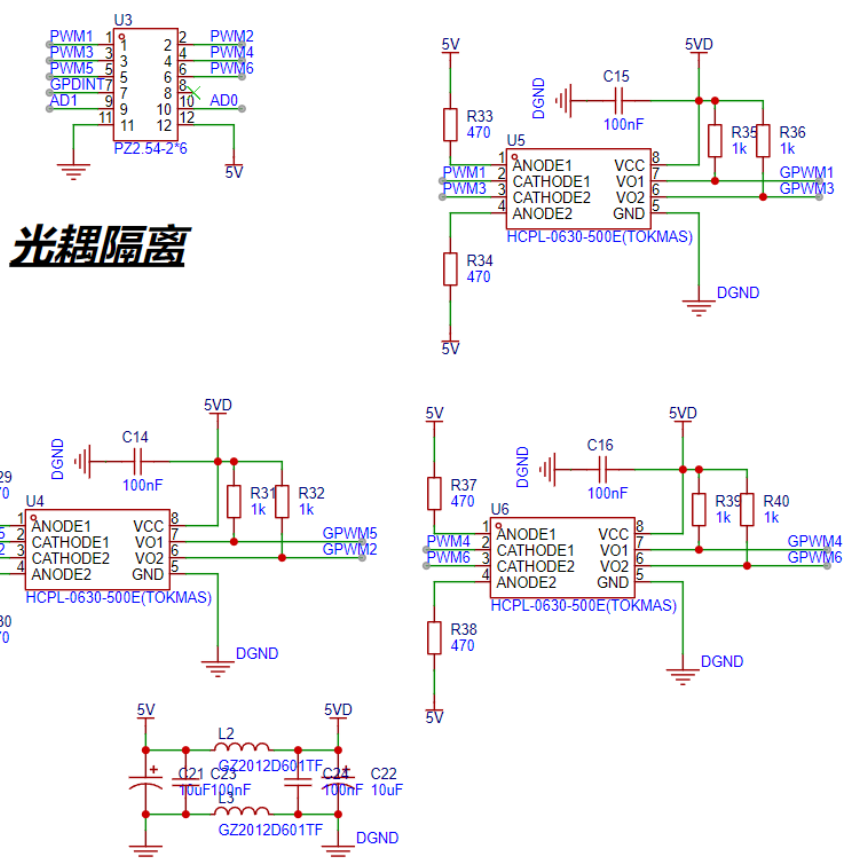
驱动部分主要包含 1. 电源(各种片上外设需要的电源轨) 2. 光耦隔离 3. 栅极驱动 4. 反相器&RC 前级 5. ADC 采样及其差分前级 6. 三相逆变电路。下面是详细的原理图

1. 电源，电源部分主要使用了一颗 LM2576S Buck 和 LM7815 线性电源，LM7815 将 24V 的电压降低为 15V，然后输入到 LM2576S Buck 降低到 5V 以供给到片上弱电系统使用。



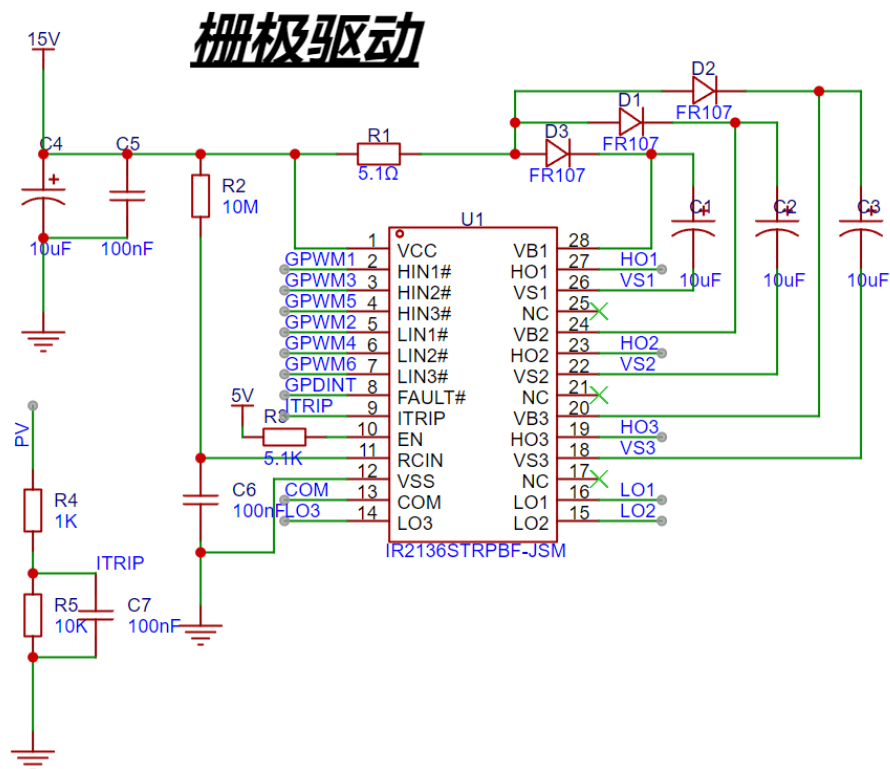
图表 10 电源部分如图

- 光耦隔离，采用三颗 HCPL-0630 对三路互补的 PWM 信号进行隔离，并且带有磁珠对电源部分进行隔离，降低噪声，防止烧毁运算单元。



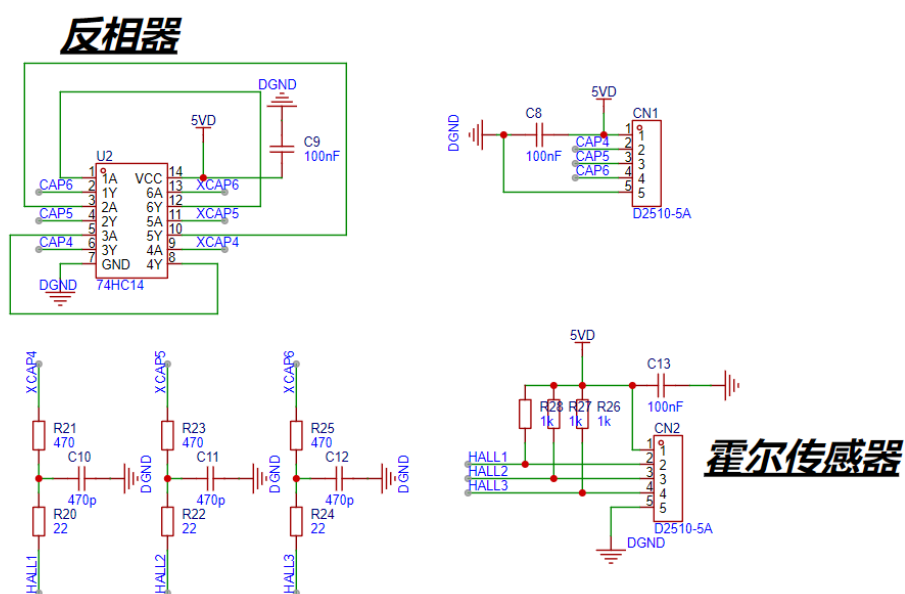
图表 11 光耦隔离

- 栅极驱动部分主要采用 IR2136S,该 IC 具有不错的灌电流。



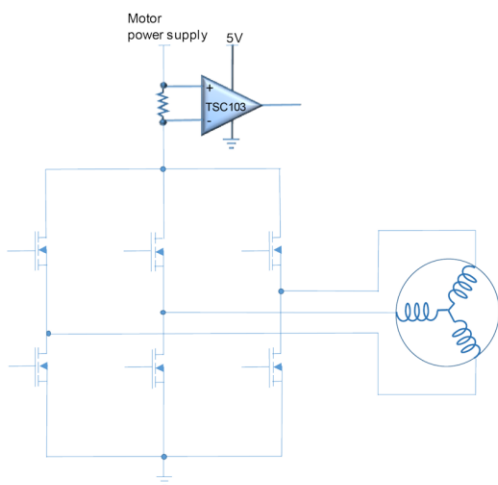
图表 12 栅极驱动

- 反相器&RC 前级，该部分主要是针对 HALL 信号而言，首先反相器对 HALL 信号进行整形，并提供缓冲，能够将缓慢变化的信号转化成急剧变换的输出信号，方便 MCU 进行捕获。虽有经过 RC 低通滤波器进行滤波，然后再输出。就完成了对 HALL 信号的缓冲于整形步骤。

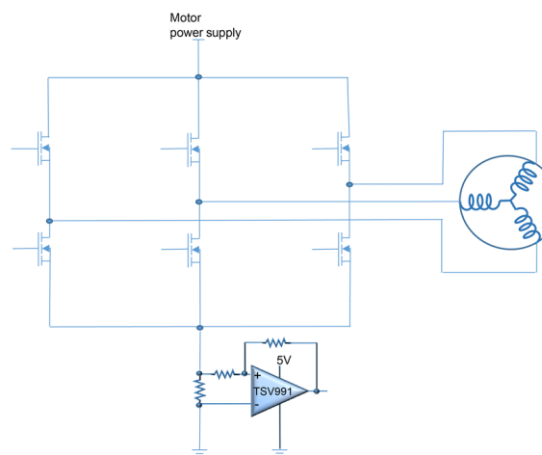


图表 13 HALL 信号的缓冲于整形

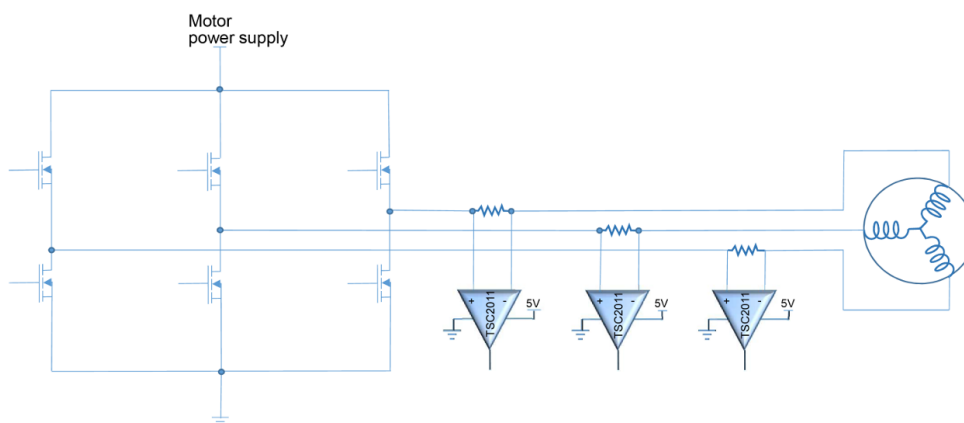
5. ADC 采样及其差分前级，采样部分根据 ST 的手册，有三种方式，分别为高侧采样，低侧采样以及相电流采样。



图表 14 高侧采样

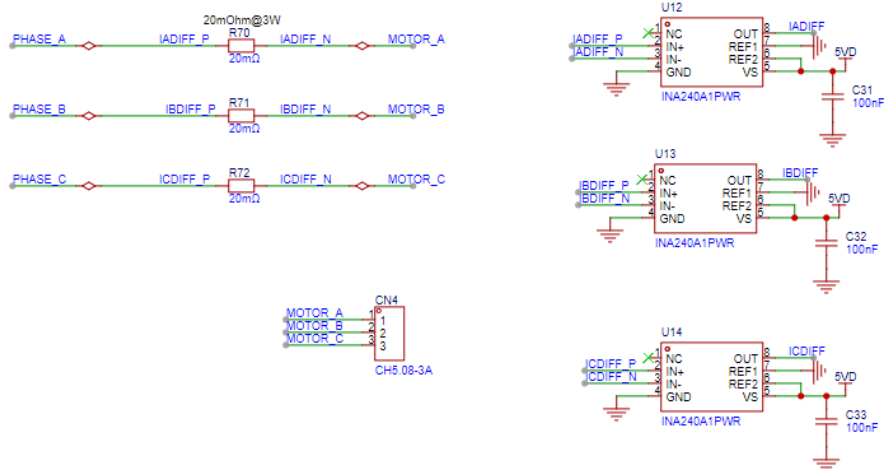


图表 15 低侧采样



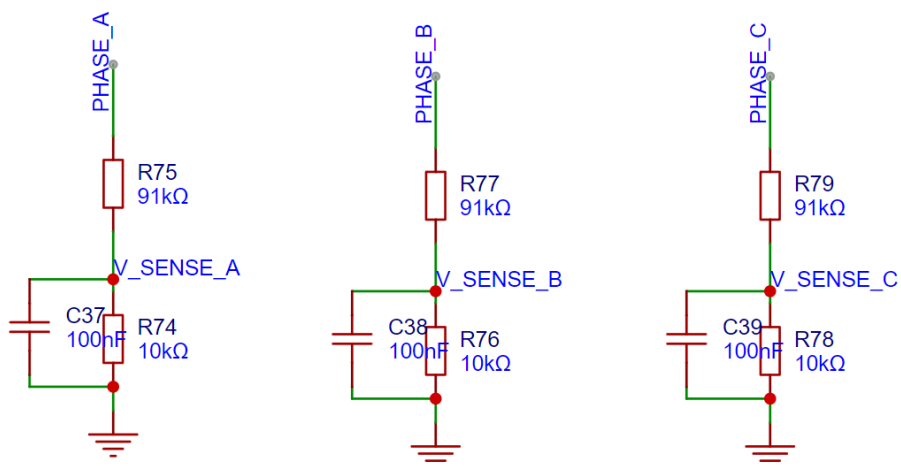
图表 16 相电流采样

本次采样直接使用相电流采样，并且使用差分运放抑制电机的共模噪声，此差分运放无需外置 RC 前级进行滤波。



图表 17 相电流差分采样

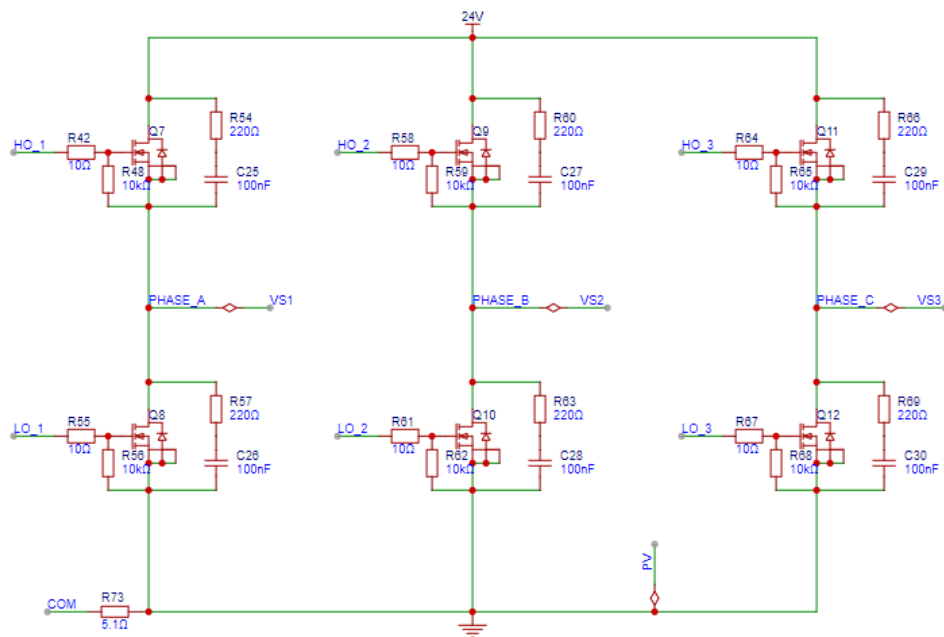
此差分采样再搭配 H7 的差分注入通道进行 ADC 采样，软件搭配 DMA 去传输数据以实现不错的采样效果，为了节省通道，这里的电压采样直接使用单端的电阻分压进行相电压采样。此数据可以用于 FOC 算法的前馈矫正。



图表 18 相电压采样

此外应当再设定一个母线电压采样用于过压检测等安全性措施。

6. 三相逆变电路，直接采用三个半桥用于构建逆变电路，后期用于 SVPWM 的调制，输出马鞍波形。



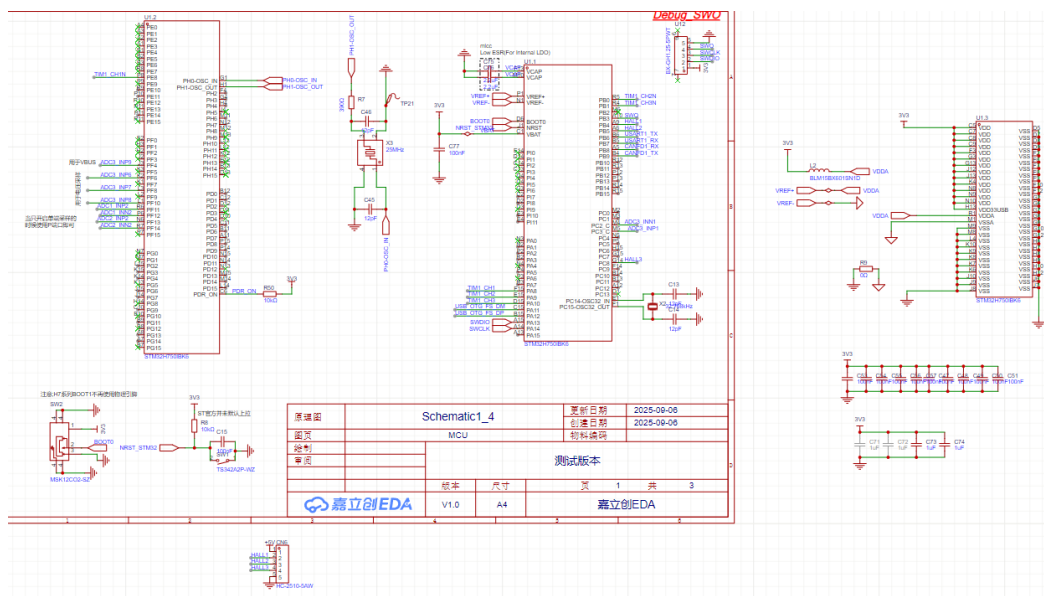
图表 19 逆变电路

此时 MOS 配备有 栅极驱动电阻，RC 吸收电路，下拉电阻，分别用于调整栅极驱动的信号震铃以及限制栅极电流，RC 吸收电路通常用于减小 MOSFET 开关时产生的高频噪声和反向电压，下拉电阻用于确保 MOSFET 在没有驱动信号时，栅极电压稳定在零伏（或接地）。这有助于防止 MOSFET 误导通或保持导通状态。

3.2 控制部分

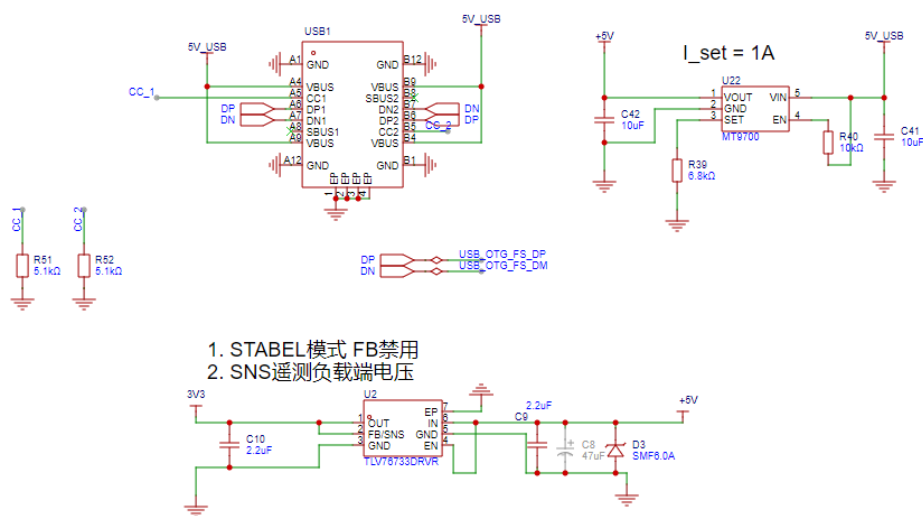
控制部分，本项目主要采用了 MCU,使用 STM32H750IBK6 作为控制核心。控制部分主要有 1. 最小系统 2. 电源 3. 接口

1. 最小系统，最小系统只需要给出 STM32H7 的完整电源轨，以及配置时钟和 boot 电路和 debug 电路即可



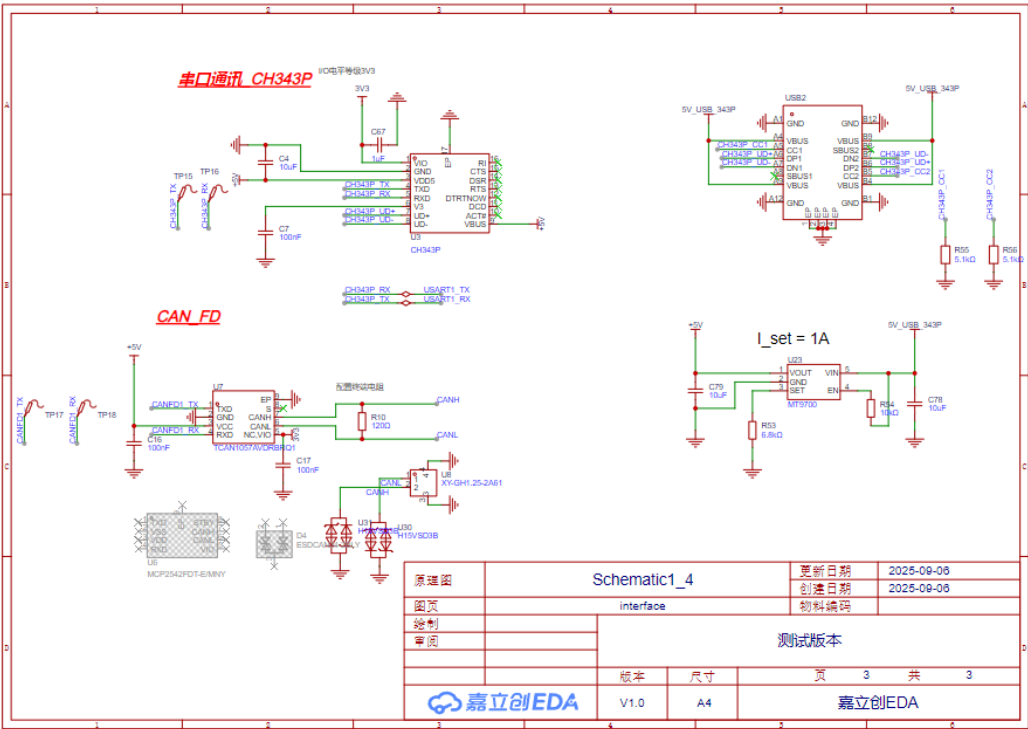
图表 20 控制最小系统

2. 电源部分主要就是两路的 5V 输入使用 MT9700 进行复用然后经过一颗 LDO 输出 3V3 即完成了电源轨的配置



图表 21 电源部分

3. 接口部分主要包含了 1. USB OGT 2. 串口 3. CAN 4. Debug
- USB OTG 直连 STM32H7 内部，用于传递信息，串口用于输出阶段性信息，CAN 接口主要用于上位控制电机的各项参数。Debug 接口包含了 SWD+SWO 用于 Trace debug，并且配置 RTT 用于大量的实时性日志级别调试。



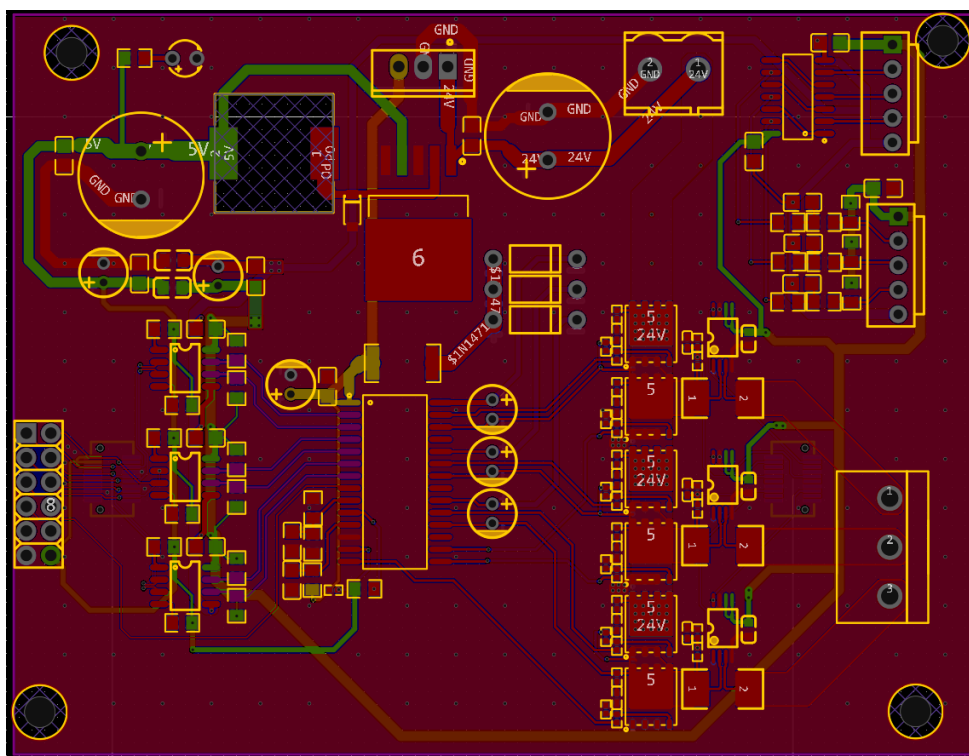
图表 22 接口部分

此外 ADC 采样直接通过 BTB，差分信号可以包地处理。

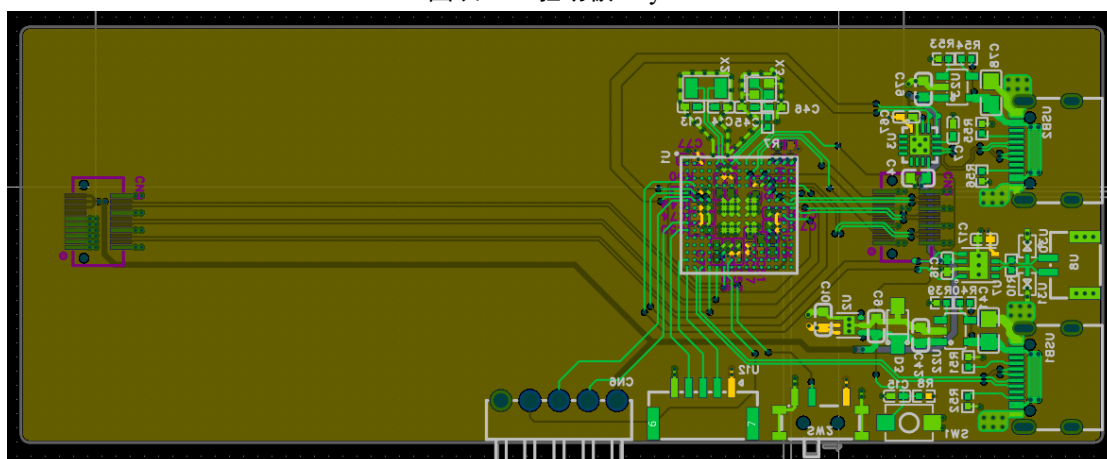


图表 23 ADC 采样

3.3 PCB Layout



图表 24 驱动板 Layout



图表 25 控制板 Layout

第 4 章 软件设计

- 软件工程已开源至 GitHub: [Ashington258/BLDC_PMSM_Controller](https://github.com/Ashington258/BLDC_PMSM_Controller)

该项目使用 Cmake 进行构建，使用 HAL 库作为底层驱动。

主要包含 1. 系统初始化及其 BSP 配置 2. 控制代码

4.1 系统配置

整个系统的配置主要包含了 1. 内核及其时钟的配置 2. 各个外设的初始化及其配置 2. NVIC 中断

程序入口及其内存配置，Flash.ld 文件

```
/* Entry Point */
ENTRY(Reset_Handler)

/* Highest address of the user mode stack */
_estack = ORIGIN(DTCMRAM) + LENGTH(DTCMRAM); /* end of RAM */
/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200; /* required amount of heap */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Specify the memory areas */
MEMORY
{
DTCMRAM (xrw)      : ORIGIN = 0x20000000, LENGTH = 128K
RAM (xrw)          : ORIGIN = 0x24000000, LENGTH = 512K
RAM_D2 (xrw)       : ORIGIN = 0x30000000, LENGTH = 288K
RAM_D3 (xrw)       : ORIGIN = 0x38000000, LENGTH = 64K
ITCMRAM (xrw)      : ORIGIN = 0x00000000, LENGTH = 64K
FLASH (rx)         : ORIGIN = 0x80000000, LENGTH = 128K
}
```

主要的时钟配置

```
void MX_TIM1_Init(void)
{
    /* USER CODE BEGIN TIM1_Init 0 */

    /* USER CODE END TIM1_Init 0 */

    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};
    TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};

    /* USER CODE BEGIN TIM1_Init 1 */

    /* USER CODE END TIM1_Init 1 */
    htim1.Instance = TIM1;
    htim1.Init.Prescaler = 0;
```



```
htim1.Init.CounterMode = TIM_COUNTERMODE_CENTERALIGNED1;
htim1.Init.Period = 5000-1;
htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim1.Init.RepetitionCounter = 0;
htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) !=
HAL_OK)
{
    Error_Handler();
}
if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterOutputTrigger2 = TIM_TRGO2_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim1,
&sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 2500;
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC,
TIM_CHANNEL_1) != HAL_OK)
{
    Error_Handler();
}
if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC,
TIM_CHANNEL_2) != HAL_OK)
{
    Error_Handler();
}
```

```

    if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC,
TIM_CHANNEL_3) != HAL_OK)
    {
        Error_Handler();
    }
    sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_ENABLE;
    sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_ENABLE;
    sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
    sBreakDeadTimeConfig.DeadTime = 200;
    sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
    sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
    sBreakDeadTimeConfig.BreakFilter = 0;
    sBreakDeadTimeConfig.Break2State = TIM_BREAK2_DISABLE;
    sBreakDeadTimeConfig.Break2Polarity = TIM_BREAK2POLARITY_HIGH;
    sBreakDeadTimeConfig.Break2Filter = 0;
    sBreakDeadTimeConfigAutomaticOutput =
TIM_AUTOMATICOUTPUT_ENABLE;
    if (HAL_TIMEx_ConfigBreakDeadTime(&htim1,
&sBreakDeadTimeConfig) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM1_Init 2 */

    // 启动三路主通道 + 三路互补通道
    if (HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1) != HAL_OK)
        Error_Handler();
    if (HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1) != HAL_OK)
        Error_Handler();
    if (HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2) != HAL_OK)
        Error_Handler();
    if (HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_2) != HAL_OK)
        Error_Handler();
    if (HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3) != HAL_OK)
        Error_Handler();
    if (HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_3) != HAL_OK)
        Error_Handler();
    // 开启 TIM1 基本定时中断（会置位 DIER.UIE）
    HAL_TIM_Base_Start_IT(&htim1);
    /* USER CODE END TIM1_Init 2 */
    HAL_TIM_MspPostInit(&htim1);

}

```

主要的中断源

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    // 开环控制
    if (htim->Instance == TIM1)
    {
        SVPWM_TaskStep(&svpwm); // 与 PWM 更新同步, 20 kHz
    }
    // 闭环控制
    // if (htim->Instance == TIM1)
    // {

    // /* 1) 更新时间戳 & 霍尔插值角度 (电角度) */
    // uint32_t now_us = HAL_GetTick() * 1000u;
    // Hall_UpdateAngle(&g_hall, now_us);
    // FOC_UpdateAngle(&foc, g_hall.theta_e);

    // /* 2) 更新 Vdc (可 1 kHz 读一次, 这里简化每次都读) */
    // FOC_UpdateVdc(&foc, BusVoltage_Read());

    // /* 3) 速度外环 (分频执行), 将 rpm_ref → iq_ref */
    // if (FOC_ShouldRunSpeedLoop(&foc))
    // {
    //     FOC_StepSpeedLoop(&foc);
    // }

    // /* 4) 电流环一步: 得到 v_alpha/v_beta */
    // FOC_StepCurrentLoop(&foc);

    // /* 5) 送入 SVPWM: 这里让 SVPWM 工作在 αβ 模式 */
    // /* 归一化: 把相电压 (V) → [-1,1], 用估计的 Vdc 映射 */
    // float scale = (foc.vdc > 1.f) ? (1.0f / foc.vdc) : 0.0f;
    // SVPWM_SetAlphaBeta(&svpwm, foc.v_alpha * scale,
    foc.v_beta * scale);
    // SVPWM_Step(&svpwm);
    // }
}

extern HallObs g_hall;
/* 读取 UVW (你用的 TIM3_CH1/2/3 是 PB4/PB5/PC8), AF 输入同样能读
GPIO IDR */
static inline uint8_t Hall_ReadUVW(void)
{
    uint8_t u = (uint8_t)HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4);
```

```

uint8_t v = (uint8_t)HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_5);
uint8_t w = (uint8_t)HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_8);
return (uint8_t)((u ? 1 : 0) | (v ? 2 : 0) | (w ? 4 : 0));
}
/* 霍尔边沿来自 CC1 → HAL_TIM_IC_CaptureCallback */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3 && htim->Channel ==
HAL_TIM_ACTIVE_CHANNEL_1)
    {
        uint32_t now_us = HAL_GetTick() * 1000u; // 可换为更精准的定时器
计数
        Hall_OnEdge(&g_hall, Hall_ReadUVW(), now_us);
    }
}

```

4.2 控制代码

控制主要包含 1. 采样部分 2. FOC 部分(解耦) 3. SVPWM 调制

采样主要使用 3 个通道的差分注入通道(用于采集电流数据), 3 个通道的 ADC 单端采样(用于电压的采集), 以及一个总线电压采样(用于电压检测, 过压欠压保护)

ADC 采样

```

static uint32_t HAL_RCC_ADC12_CLK_ENABLED=0;

void HAL_ADC_MspInit(ADC_HandleTypeDef* adcHandle)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    if(adcHandle->Instance==ADC1)
    {
        /* USER CODE BEGIN ADC1_MspInit 0 */

        /* USER CODE END ADC1_MspInit 0 */
        /* ADC1 clock enable */
        HAL_RCC_ADC12_CLK_ENABLED++;
        if(HAL_RCC_ADC12_CLK_ENABLED==1){
            __HAL_RCC_ADC12_CLK_ENABLE();
        }

        __HAL_RCC_GPIOF_CLK_ENABLE();
    }
}

```

```
/**ADC1 GPIO Configuration
PF11      -----> ADC1_INP2
*/
GPIO_InitStruct.Pin = GPIO_PIN_11;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOF, &GPIO_InitStruct);

/* USER CODE BEGIN ADC1_MspInit 1 */

/* USER CODE END ADC1_MspInit 1 */
}
else if(adcHandle->Instance==ADC2)
{
/* USER CODE BEGIN ADC2_MspInit 0 */

/* USER CODE END ADC2_MspInit 0 */
/* ADC2 clock enable */
HAL_RCC_ADC12_CLK_ENABLED++;
if(HAL_RCC_ADC12_CLK_ENABLED==1){
    __HAL_RCC_ADC12_CLK_ENABLE();
}

__HAL_RCC_GPIOF_CLK_ENABLE();
/**ADC2 GPIO Configuration
PF13      -----> ADC2_INP2
*/
GPIO_InitStruct.Pin = GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOF, &GPIO_InitStruct);

/* USER CODE BEGIN ADC2_MspInit 1 */

/* USER CODE END ADC2_MspInit 1 */
}
else if(adcHandle->Instance==ADC3)
{
/* USER CODE BEGIN ADC3_MspInit 0 */

/* USER CODE END ADC3_MspInit 0 */
/* ADC3 clock enable */
__HAL_RCC_ADC3_CLK_ENABLE();
```

```

    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();
    /**ADC3 GPIO Configuration
    PF4      -----> ADC3_INP9
    PF6      -----> ADC3_INP8
    PF10     -----> ADC3_INP6
    PF8      -----> ADC3_INP7
    PC3_C    -----> ADC3_INP1
    */
    GPIO_InitStruct.Pin =
GPIO_PIN_4|GPIO_PIN_6|GPIO_PIN_10|GPIO_PIN_8;
    GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOF, &GPIO_InitStruct);

    HAL_SYSCFG_AnalogSwitchConfig(SYSCFG_SWITCH_PC3,
SYSCFG_SWITCH_PC3_OPEN);

    /* USER CODE BEGIN ADC3_MspInit 1 */

    /* USER CODE END ADC3_MspInit 1 */
}
}

void HAL_ADC_MspDeInit(ADC_HandleTypeDef* adcHandle)
{

    if(adcHandle->Instance==ADC1)
    {
        /* USER CODE BEGIN ADC1_MspDeInit 0 */

        /* USER CODE END ADC1_MspDeInit 0 */
        /* Peripheral clock disable */
        HAL_RCC_ADC12_CLK_ENABLED--;
        if(HAL_RCC_ADC12_CLK_ENABLED==0){
            __HAL_RCC_ADC12_CLK_DISABLE();
        }

        /**ADC1 GPIO Configuration
        PF11     -----> ADC1_INP2
        */
        HAL_GPIO_DeInit(GPIOF, GPIO_PIN_11);

        /* USER CODE BEGIN ADC1_MspDeInit 1 */

```

```

/* USER CODE END ADC1_MspDeInit 1 */
}
else if(adcHandle->Instance==ADC2)
{
/* USER CODE BEGIN ADC2_MspDeInit 0 */

/* USER CODE END ADC2_MspDeInit 0 */
/* Peripheral clock disable */
HAL_RCC_ADC12_CLK_ENABLED--;
if(HAL_RCC_ADC12_CLK_ENABLED==0){
__HAL_RCC_ADC12_CLK_DISABLE();
}

/**ADC2 GPIO Configuration
PF13      -----> ADC2_INP2
*/
HAL_GPIO_DeInit(GPIOF, GPIO_PIN_13);

/* USER CODE BEGIN ADC2_MspDeInit 1 */

/* USER CODE END ADC2_MspDeInit 1 */
}
else if(adcHandle->Instance==ADC3)
{
/* USER CODE BEGIN ADC3_MspDeInit 0 */

/* USER CODE END ADC3_MspDeInit 0 */
/* Peripheral clock disable */
__HAL_RCC_ADC3_CLK_DISABLE();

/**ADC3 GPIO Configuration
PF4       -----> ADC3_INP9
PF6       -----> ADC3_INP8
PF10      -----> ADC3_INP6
PF8       -----> ADC3_INP7
PC3_C     -----> ADC3_INP1
*/
HAL_GPIO_DeInit(GPIOF,
GPIO_PIN_4|GPIO_PIN_6|GPIO_PIN_10|GPIO_PIN_8);

/* USER CODE BEGIN ADC3_MspDeInit 1 */

/* USER CODE END ADC3_MspDeInit 1 */

```

```

    }
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

```

FOC 部分主要实现 1. 三相电流的采样, HALL 传感器获取转子位置/速度
2. 坐标变换(Clarke 变换和 Park 变换) 3. 控制环路(速度外环, 电流内环)

后期经过逆 Park 变换回到 α, β 两相静止坐标系生成三相 PWM 占空比, 即 SVPWM 调制。

FOC.c

```

#include "foc.h"
#include <math.h>

/* ===== 工具 ===== */
#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif
static inline float clampf(float x, float lo, float hi) { return
(x < lo) ? lo : ((x > hi) ? hi : x); }

/* Clarke (A 相对 a) */
static inline void clarke(float ia, float ib, float ic, float
*a, float *b)
{
    (void)ic; // ia+ib+ic≈0 时, 可只用 ia,ib
    *a = ia;
    *b = (ia + 2.0f * ib) * (-0.5773502691896258f); // -1/√3
}

/* Park */
static inline void park(float a, float b, float s, float c,
float *d, float *q)
{
    *d = c * a + s * b;
    *q = -s * a + c * b;
}

/* 逆 Park */
static inline void inv_park(float d, float q, float s, float c,
float *a, float *b)

```



```

{
    *a = c * d - s * q;
    *b = s * d + c * q;
}

/* PI (带积分抗风 up) */
static inline float pi_step(float err, float kp, float ki, float
*inte, float out_min, float out_max, float dt)
{
    float u = kp * err + (*inte);
    float sat = clampf(u, out_min, out_max);
    /* 仅在未饱和或有利于脱饱和方向时积分 */
    float anti = (sat == u) ? 1.0f : 0.0f;
    *inte += anti * (ki * err * dt);
    return sat;
}

/* ===== API ===== */
void FOC_Init(FOC_Ctrl *f, float fs_hz, uint8_t pole_pairs,
float vdc)
{
    f->dt_s = 1.0f / ((fs_hz > 0) ? fs_hz : 20000.0f);
    f->decim_spd = 20;
    f->cnt_spd = 0; // 每 20 次电流环 (1 kHz) 走一次速度环
    f->pole_pairs = pole_pairs;
    f->vdc = vdc;
    f->rpm_ref = 0.0f;
    f->rpm_meas = 0.0f;

    /* 经验 PI (起步值, 后续可整定) */
    f->kp_id = 0.5f;
    f->ki_id = 200.0f;
    f->kp_iq = 0.5f;
    f->ki_iq = 200.0f;
    f->kp_spd = 0.02f;
    f->ki_spd = 0.8f;

    f->int_id = f->int_iq = f->int_spd = 0.0f;
    f->id_ref = 0.0f;
    f->iq_ref = 0.0f;
    f->id_meas = f->iq_meas = 0.0f;
    f->vd_cmd = f->vq_cmd = 0.0f;

    /* 相电压限幅 ~ 0.577*Vdc (SVPWM 最大线电压→相电压) 再留裕量 */

```

```

    f->v_lim = 0.5f * vdc;

    f->theta_e = 0.0f;
    f->sin_th = 0.0f;
    f->cos_th = 1.0f;
    f->v_alpha = 0.0f;
    f->v_beta = 0.0f;
}

void FOC_SetSpeedRef(FOC_Ctrl *f, float rpm) { f->rpm_ref = rpm; }
void FOC_UpdateAngle(FOC_Ctrl *f, float theta_e)
{
    f->theta_e = theta_e;
    f->sin_th = sinf(theta_e);
    f->cos_th = cosf(theta_e);
}
void FOC_UpdateVdc(FOC_Ctrl *f, float vdc)
{
    if (vdc > 1.f)
    {
        f->vdc = vdc;
        f->v_lim = 0.5f * vdc;
    }
}

int FOC_ShouldRunSpeedLoop(FOC_Ctrl *f) { return (++f->cnt_spd >= f->decim_spd); }

void FOC_StepSpeedLoop(FOC_Ctrl *f)
{
    f->cnt_spd = 0;
    /* 速度反馈（来自霍尔的电角频率→机械 rpm） */
    f->rpm_meas = SpeedFeedback_FromHall_RPM();
    float err = f->rpm_ref - f->rpm_meas;
    /* 输出 Iq_ref（电流单位安培）——注意后续你要根据电机额定电流调整限幅 */
    float iq_ref = pi_step(err, f->kp_spd, f->ki_spd,
    &f->int_spd, -10.0f, +10.0f, f->dt_s * f->decim_spd);
    f->iq_ref = iq_ref;
    f->id_ref = 0.0f; /* 弱磁先不做 */
}

void FOC_StepCurrentLoop(FOC_Ctrl *f)

```

```

{
    /* 1) 读取三相电流 (A) */
    float ia = 0, ib = 0, ic = 0;
    PhaseCurrent_Read_ABC(&ia, &ib, &ic);

    /* 2) Clarke → Park */
    float a, b;
    clarke(ia, ib, ic, &a, &b);
    float d, q;
    park(a, b, f->sin_th, f->cos_th, &d, &q);
    f->id_meas = d;
    f->iq_meas = q;

    /* 3) Id, Iq 双 PI (单位: 电压) */
    float vd = pi_step(f->id_ref - d, f->kp_id, f->ki_id,
    &f->int_id, -f->v_lim, +f->v_lim, f->dt_s);
    float vq = pi_step(f->iq_ref - q, f->kp_iq, f->ki_iq,
    &f->int_iq, -f->v_lim, +f->v_lim, f->dt_s);

    /* 4) 逆 Park → αβ, 相电压限幅 (圆限幅) */
    float va, vb;
    inv_park(vd, vq, f->sin_th, f->cos_th, &va, &vb);
    float mag = sqrtf(va * va + vb * vb);
    float vmax = 0.577f * f->vdc; // SVPWM 可用相电压峰值
    if (mag > vmax && mag > 1e-6f)
    {
        float k = vmax / mag;
        va *= k;
        vb *= k;
    }

    f->v_alpha = va;
    f->v_beta = vb;
}

/* ===== 弱符号: 把你的 ADC/Hall 接上即可 ===== */
__attribute__((weak)) void PhaseCurrent_Read_ABC(float *ia,
float *ib, float *ic)
{
    /* TODO: 换成你的 ADC 结果 (单位安培, 去偏置, 方向一致性) */
    *ia = 0.0f;
    *ib = 0.0f;
    *ic = 0.0f;
}

```

```

__attribute__((weak)) float BusVoltage_Read(void)
{
    /* TODO: 读取母线电压（例如 ADC 量测换算） */
    return 24.0f;
}

__attribute__((weak)) float SpeedFeedback_FromHall_RPM(void)
{
    /* TODO: 可直接返回你的霍尔模块计算的机械 rpm */
    return 0.0f;
}

```

4. SVPWM 调制部分，通过先前 FOC 的 dq 数据，逆 Park 变换回到 α, β 坐标系，然后根据 8 个开关矢量，加上不同的 Δt 作用，合成任意空间矢量，从而合成接近圆形的旋转磁场。

Svpwm.c

```

#include "spwm.h"
#include <math.h>

// 如果你的编译器没有 M_PI，可以自行定义：
#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif

// 角度归一化到 [0, 2π)
static inline float wrap_angle_0_2pi(float x)
{
    const float two_pi = 2.0f * (float)M_PI;
    while (x >= two_pi)
        x -= two_pi;
    while (x < 0.0f)
        x += two_pi;
    return x;
}

// 饱和
static inline float clampf(float x, float lo, float hi)
{
    return (x < lo) ? lo : (x > hi ? hi : x);
}

void SPWM_Init(SPWM_t *spwm,

```

```

        TIM_HandleTypeDef *htim,
        float update_rate_hz,
        float init_theta_rad)
{
    spwm->htim = htim;
    spwm->theta = wrap_angle_0_2pi(init_theta_rad);
    spwm->elec_freq_hz = 0.0f;
    spwm->modulation = 0.0f;
    // dt = 1 / 调用频率
    if (update_rate_hz <= 0.0f)
        update_rate_hz = 10000.0f; // fallback: 10kHz
    spwm->dt_s = 1.0f / update_rate_hz;
}

void SPWM_SetOpenloop(SPWM_t *spwm,
                      float elec_freq_hz,
                      float modulation)
{
    spwm->elec_freq_hz = elec_freq_hz;
    spwm->modulation = clampf(modulation, 0.0f, 0.98f);
}

/**
 * 正弦开环 (SPWM):
 *  Va = m * sin(θ)
 *  Vb = m * sin(θ - 2π/3)
 *  Vc = m * sin(θ + 2π/3)
 *  映射到占空比:
 *  duty = 0.5 + 0.5 * Va    (保证在[0,1]内, 当 m ≤ 1)
 *  对应 CCR = duty * ARR
 *
 *  注意: 互补与死区由定时器硬件处理, 这里只写 CCR1/2/3.
 */
void SPWM_Update(SPWM_t *spwm)
{
    if (!spwm || !spwm->htim)
        return;

    // 1) 推进电角度:  $\theta(k+1) = \theta(k) + 2\pi * f_{elec} * dt$ 
    spwm->theta += 2.0f * (float)M_PI * spwm->elec_freq_hz *
    spwm->dt_s;
    spwm->theta = wrap_angle_0_2pi(spwm->theta);

    // 2) 生成三相正弦 (电压归一化)

```

```

const float m = spwm->modulation; // 0..0.98
const float th = spwm->theta;
const float th_b = th - 2.0f * (float)M_PI / 3.0f;
const float th_c = th + 2.0f * (float)M_PI / 3.0f;

const float va = m * sinf(th);
const float vb = m * sinf(th_b);
const float vc = m * sinf(th_c);

// 3) 归一化到占空比 [0,1]
float du_a = 0.5f + 0.5f * va;
float du_b = 0.5f + 0.5f * vb;
float du_c = 0.5f + 0.5f * vc;

du_a = clampf(du_a, 0.0f, 1.0f);
du_b = clampf(du_b, 0.0f, 1.0f);
du_c = clampf(du_c, 0.0f, 1.0f);

// 4) 写入 CCR (与计数模式无关; 中心对齐时 HAL 内部会处理翻转)
uint32_t arr = __HAL_TIM_GET_AUTORELOAD(spwm->htim); // 例如
4999
uint32_t ccr1 = (uint32_t)(du_a * (float)arr);
uint32_t ccr2 = (uint32_t)(du_b * (float)arr);
uint32_t ccr3 = (uint32_t)(du_c * (float)arr);

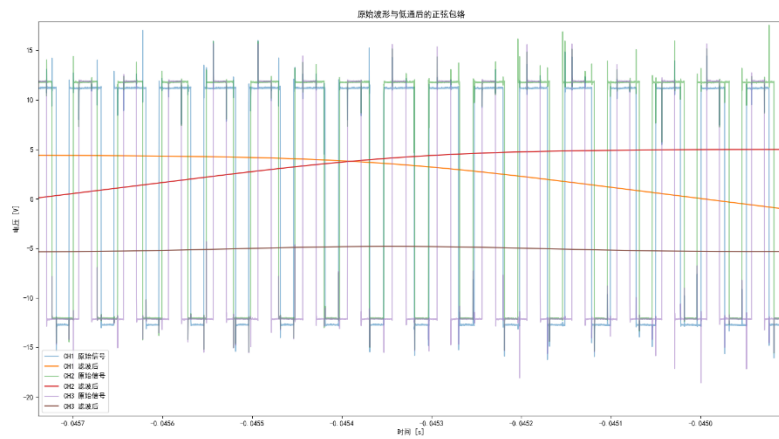
__HAL_TIM_SET_COMPARE(spwm->htim, TIM_CHANNEL_1, ccr1);
__HAL_TIM_SET_COMPARE(spwm->htim, TIM_CHANNEL_2, ccr2);
__HAL_TIM_SET_COMPARE(spwm->htim, TIM_CHANNEL_3, ccr3);
}

```

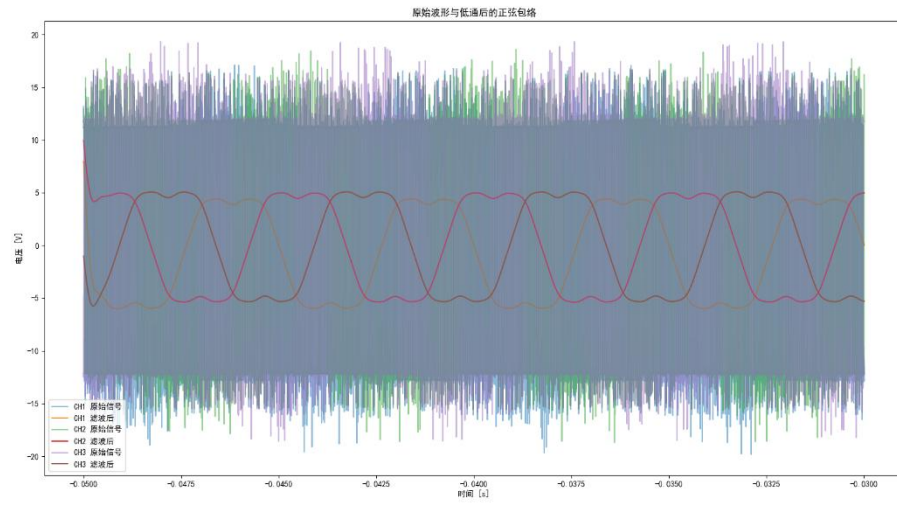
第5章 测试结果分析

本次测试主要通过示波器采集数据, 转化为 scv 文件, 使用 python 读取以及使用 matplotlib 进行分析。

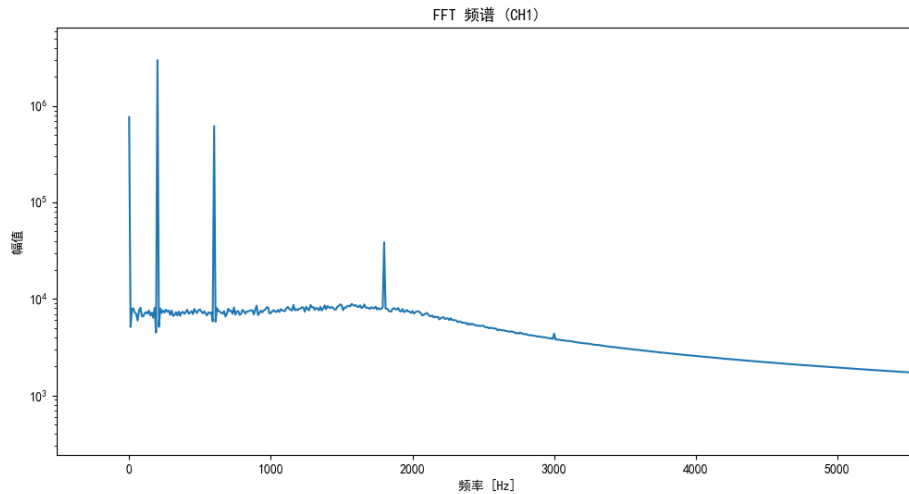
首先获取栅极驱动信号, 三通道采样, 根据软件计算三个相位 (0.0° , 120.2° , 240.2°)



图表 26 栅极驱动信号



图表 27 马鞍波



图表 28 频谱分析

第6章 设计总结

首先，本次测试发现采样部分设计不够完善 1. 为设计硬件钳位 2. ADC 灵敏度不佳，应该改用更大增益的差分运放以及对 ADC 端口增加硬件钳位保护 MCU 的 IO。其次，本次未引入 BKIN 机制，即刹车输入，Break Input，应当设计一个电压比较器，设定阈值，当过流时触发 break 信号直接输入 STM32 的 Break Input 通道，关闭所有的 PWM 通道进行刹车处理。最后，本次设计的安全设计仍有不足，例如未设计全面的看门狗机制，硬件防护机制。

改进部分，MCU 可以更换为 FPGA 用于跟进最新的 FOC 无感控制算法，结合 FPGA 的优秀硬件加速能力以及 ARM 核的处理能力，选用 ZYNQ 系列。